CSE310: Compiler Sessional, January 2026 Term

ICG Online Assignment (B1)

Printing Multiple Expressions

Time: 40 Mins Total Marks: 10

Problem

In this online, you are required to extend the grammar to support **printing values of multiple expressions with a single println statement**. Thus, your program should support the following syntax.

```
int a, b;
a=2; b=3;
println(a+b, 2+3*4, a*2+3); //printing of multiple comma separated expressions
```

Task

1. Extend the current grammar to support **printing values of multiple expressions with a single println statement**. The expressions will be comma (,) separated. Like the offline, you need to work only with integer type variables. There will be no syntax or semantic errors in the given input program.
2. Update your ICG code to generate assembly code for this new rule/syntax. The assembly code output by your program must correctly be assembled with 'fasm' . Subsequently, the binary produced by 'fasm' from your assembly code must produce the expected output for the input program.

Mark Distribution

- Extending the grammar to support printing values of multiple expressions. (5)
- Code generation for printing values of multiple expressions. (5)

Find a sample input-output in the next page.

Sample Input-Output

Input Program

```
1 int a, b;
2 int main(){
3     int x, y;
4     a = 10;
5     b = 20;
6     x = 100;
7     y = 200;
8     println(a, b, x, y);
9     println(a*2+b, x+y-10, 3+3*7);
10    println(a==b, a&&b, a++, -a);
11    return 0;
12 }
```

Expected Output (from the generated binary by fasm)

```
10
20
100
200
40
290
24
0
1
10
-11
```

Generated Sample Assembly (print library omitted)

```
format ELF executable 3
entry main
segment readable writeable
      a dd 1 DUP (0)
      b dd 1 DUP (0)
segment readable executable
main:
      PUSH EBP
      MOV EBP, ESP
      SUB ESP, 4
      SUB ESP, 4
.L1:
      MOV EAX, 10        ; Line 4
      MOV [a], EAX
.L2:
      MOV EAX, 20        ; Line 5
      MOV [b], EAX
.L3:
      MOV EAX, 100       ; Line 6
      MOV [EBP-4], EAX
.L4:
      MOV EAX, 200       ; Line 7
      MOV [EBP-8], EAX
.L5:
      MOV EAX, [a]       ; Line 8
      CALL print_number
      MOV EAX, [b]       ; Line 8
      CALL print_number
      MOV EAX, [EBP-4]       ; Line 8
      CALL print_number
      MOV EAX, [EBP-8]       ; Line 8
      CALL print_number
.L6:
      MOV EAX, 2       ; Line 9
      MOV ECX, EAX
      MOV EAX, [a]       ; Line 9
      CWD
      MUL ECX
      PUSH EAX
      MOV EAX, [b]       ; Line 9
      MOV EDX, EAX
      POP EAX       ; Line 9
      ADD EAX, EDX
      CALL print_number
      MOV EAX, [EBP-8]       ; Line 9
      MOV EDX, EAX
      MOV EAX, [EBP-4]       ; Line 9
      ADD EAX, EDX
      PUSH EAX
```

```
        MOV EAX, 10         ; Line 9
        MOV EDX, EAX
        POP EAX         ; Line 9
        SUB EAX, EDX
        CALL print_number
        MOV EAX, 7          ; Line 9
        MOV ECX, EAX
        MOV EAX, 3          ; Line 9
        CWD
        MUL ECX
        MOV EDX, EAX
        MOV EAX, 3          ; Line 9
        ADD EAX, EDX
        CALL print_number
.L7:
        MOV EAX, [b]        ; Line 10
        MOV EDX, EAX
        MOV EAX, [a]        ; Line 10
        CMP EAX, EDX
        JE .L8
        JMP .L10
.L8:
        MOV EAX, 1          ; Line 10
        JMP .L9
.L10:
        MOV EAX, 0
.L9:
        CALL print_number
        MOV EAX, [a]        ; Line 10
        CMP EAX, 0
        JNE .L11
        JMP .L14
.L11:
        MOV EAX, [b]        ; Line 10
        CMP EAX, 0
        JNE .L12
        JMP .L14
.L12:
        MOV EAX, 1          ; Line 10
        JMP .L13
.L14:
        MOV EAX, 0
.L13:
        CALL print_number
        MOV EAX, [a]        ; Line 10
        PUSH EAX
        INC EAX
        MOV [a], EAX
        POP EAX         ; Line 10
        CALL print_number
        MOV EAX, [a]        ; Line 10
        NEG EAX
        CALL print_number
.L15:
```

	`` MOV EAX, 0 ; Line 11 JMP .L17 .L16: .L17: ADD ESP, 8 POP EBP MOV EAX,1 XOR EBX, EBX INT 0x80 POP EBP RET ```